Adding a Security Access ($27) Module

Contents

Architecture in 30 seconds
The example: GM E38
Step 1 - Implement ISeedKeyAlgorithm
Step 2 - Register the module
Step 3 - Test the algorithm
Step 4 - Try it from the UI
Step 5 - Going deeper
  Multi-level algorithms
  Different seed / key byte counts
  Stateful or derived-key algorithms
  Real-world test vectors
What Gmw3110_2010_Generic does for you
Escape hatch - implementing ISecurityAccessModule directly
Reference
  NRC quick table
  NodeState security fields
  File layout
A note on responsible use

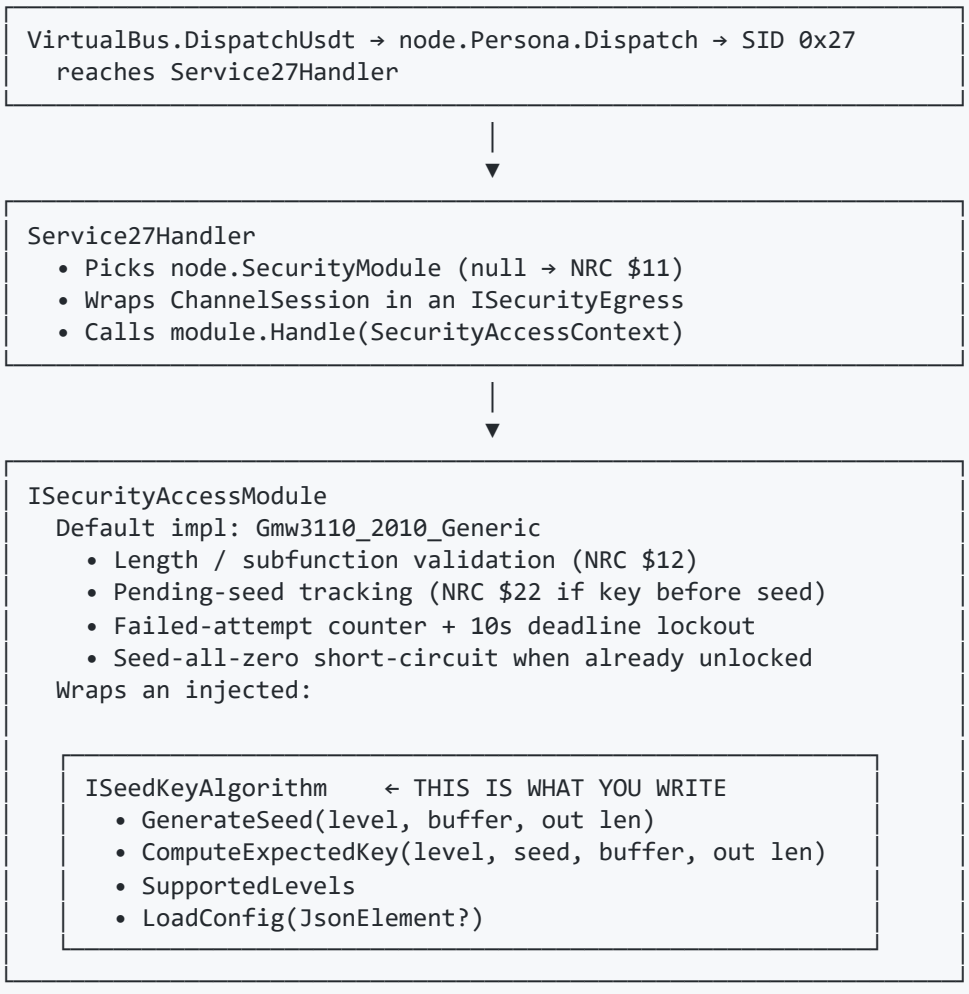
This walkthrough shows how a `$27` module is built, using GM's E38 ECM seed-key algorithm as the worked example. E38 already ships in the simulator as the `gm-e38-2byte` module - this doc dissects that real code so you can follow the same pattern to add your own GM seed-key flavour.

By the end you'll understand:

- The C# class that implements an algorithm (`E38Algorithm`)
- The one-line registry entry the UI picks up automatically
- The xUnit tests that pin the math and prove the wire exchange unlocks

The same pattern works for any GM seed-key flavour you want to add - VATS, the 4-byte schemes on newer ECUs, MAC-based exchanges. All you change is the algorithm. (The simulator already ships modules for E38, E67, the GM 5-byte DPS cipher, and T43 - see `SecurityModuleRegistry` .)

Architecture in 30 seconds



Two interfaces, two layers of swap-in. 90% of the time you write an `ISeedKeyAlgorithm` and let `Gmw3110_2010_Generic` handle the protocol bookkeeping. The full `ISecurityAccessModule` interface exists as an escape hatch for non-standard protocol flows (multi-message exchanges, MAC-based schemes) - see §7.

The example: GM E38

The E38 ECM (mid-2000s LS-engine GM vehicles) uses GMLAN algorithm `0x92`:

- **Seed:** 2 bytes
- **Key:** 2 bytes
- **Level:** 1 only
- **Algorithm** (16-bit values, well documented in the GM tuning community):

`C k = byteSwap16(seed); k = k + 0x7D58; k = ~k; k = k & 0xFFFF; k = k + 0x8001; key = byteSwap16(k & 0xFFFF);`

What this looks like on the wire (ISO-TP Single Frames on the ECU's physical request / USDT response CAN IDs):

```
Tester → ECU   02 27 01                         // requestSeed level 1
ECU   → Tester 04 67 01 12 34                   // seed = 0x1234
Tester → ECU   04 27 02 96 CE                   // sendKey = E38(0x1234) = 0x96CE
ECU   → Tester 02 67 02                         // unlocked ✓

  // wrong key path:
Tester → ECU   04 27 02 00 00
ECU   → Tester 03 7F 27 35                      // invalidKey
  // ... three failures total → NRC $36, 10s window → NRC $37
```

Step 1 - Implement `ISeedKeyAlgorithm`

The shipped algorithm lives in `Core/Security/Algorithms/E38Algorithm.cs` (for a new one, create your own file alongside it). The interesting parts:

1.1 - declare the algorithm's shape.

```
public sealed class E38Algorithm : ISeedKeyAlgorithm
{
    public string Id => "gm-e38-2byte";
    public int SeedLength => 2;
    public int KeyLength => 2;
    public IEnumerable<byte> SupportedLevels { get; } = new byte[] { 1 };
    // ...
}
```

`SupportedLevels` is what gatekeeps unsupported subfunctions - `Gmw3110_2010_Generic` reads this list and returns NRC `$12` for any level not in it. No need to check levels in your `ComputeExpectedKey`.

1.2 - the math itself. A static method so unit tests can hit it without constructing a context:

```
public static ushort ComputeKey(ushort seed)
{
    uint k = (uint)((seed >> 8) | ((seed & 0xFF) << 8));   // byte swap
    k = k + 0x7D58;                                         // add magic
    k = ~k;                                                 // bitwise NOT
    k = k & 0xFFFF;                                         // mask to 16 bits
    k = k + 0x8001;                                         // add magic
    return (ushort)(((k & 0xFF00) >> 8) | ((k & 0xFF) << 8)); // byte swap
}
```

1.3 - wire it into `ComputeExpectedKey`. Endianness is your responsibility - `seed` arrives as the on-wire bytes in big-endian order:

```
public bool ComputeExpectedKey(byte level, ReadOnlySpan<byte> seed, Span<byte> keyBuffer,
out int keyLength)
{
    if (level != 1 || seed.Length != 2) { keyLength = 0; return false; }
    ushort s = (ushort)((seed[0] << 8) | seed[1]);
    ushort k = ComputeKey(s);
    keyBuffer[0] = (byte)(k >> 8);
    keyBuffer[1] = (byte)(k & 0xFF);
    keyLength = 2;
    return true;
}
```

Returning `false` here tells the generic module "I have no acceptable key for this seed" - it translates to NRC `$35` invalid key, which is the right semantic when something about the input was wrong.

1.4 - generate seeds. A real ECU produces a fresh random seed per request. `Random.Shared.NextBytes` is fine for a simulator. The one gotcha: avoid emitting an all-zero seed, because the generic module treats that as the "already unlocked" signal:

```
public void GenerateSeed(byte level, Span<byte> seedBuffer, out int seedLength)
{
    if (fixedSeed is not null)
    {
        fixedSeed.AsSpan().CopyTo(seedBuffer);
    }
    else
    {
        Random.Shared.NextBytes(seedBuffer.Slice(0, 2));
        if (seedBuffer[0] == 0 && seedBuffer[1] == 0) seedBuffer[0] = 1;
    }
    seedLength = 2;
}
```

The `fixedSeed` field is populated by `LoadConfig` when the user (or a test) sets `{"fixedSeed": "1234"}` in `SecurityModuleConfig`. Useful when you want a repeatable exchange.

1.5 - `LoadConfig` for the fixed-seed option:

```
public void LoadConfig(JsonElement? config)
{
    fixedSeed = null;
    if (config is null || config.Value.ValueKind != JsonValueKind.Object) return;
    if (!config.Value.TryGetProperty("fixedSeed", out var prop)) return;
    if (prop.ValueKind != JsonValueKind.String) return;
    if (TryParseHex16(prop.GetString(), out var hi, out var lo))
        fixedSeed = new[] { hi, lo };
}
```

Module-specific config travels as a `JsonElement` blob (`EcuDto.SecurityModuleConfig`). Each algorithm deserialises its own shape - `ConfigSchema` doesn't need to know about any of them. The UI's flat key/value editor stores each value as a JSON string; parse hex / numbers in `LoadConfig` as needed.

> See the full file at Core/Security/Algorithms/E38Algorithm.cs.

Step 2 - Register the module

Each module is one `Register` line in `SecurityModuleRegistry` 's static constructor. E38 is already there; a new algorithm slots in beside it the same way:

```
static SecurityModuleRegistry()
{
    // ... E67, T43, 5-byte, bypass modules ...

    Register("gm-e38-2byte",
        () => new Gmw3110_2010_Generic(new E38Algorithm(),
                                       id: "gm-e38-2byte"));

    // Your new module: one line + your ISeedKeyAlgorithm class.
    Register("gm-myecm-2byte",
        () => new Gmw3110_2010_Generic(new MyEcmAlgorithm(),
                                       id: "gm-myecm-2byte"));
}
```

The id follows the `gm-{ecmFamily}-{width}` convention for strict modules (and `gm-bypass-{width}` for permissive ones). Old ids from previous naming passes still load - `SecurityModuleRegistry.NormaliseLegacyId` remaps them to the current id at load time - so you don't break existing `.mode.json` configs by renaming.

The factory returns a new instance per ECU, so each `EcuNode` gets its own algorithm state. If your algorithm caches derived keys, that cache lives in the instance, not statically.

Step 3 - Test the algorithm

Two layers of test pay back fast: the pure-function math, and the end-to-end wire exchange via `Service27Handler`. Both live in `Tests.Unit/Security/E38AlgorithmTests.cs`.

3.1 - pin the math with test vectors.

```
[Theory]
[InlineData((ushort)0x1234, (ushort)0x96CE)]
[InlineData((ushort)0xA1B2, (ushort)0x0750)]
[InlineData((ushort)0xDEAD, (ushort)0xCA54)]
[InlineData((ushort)0xCAFE, (ushort)0xDE03)]
[InlineData((ushort)0xFFFF, (ushort)0xA902)]
public void ComputeKey_MatchesDocumentedVectors(ushort seed, ushort expectedKey)
{
    Assert.Equal(expectedKey, E38Algorithm.ComputeKey(seed));
}
```

If you have a real-world capture (seed/key pair logged from a J2534 host talking to physical hardware), add it as another `[InlineData]`. That's the single most valuable test you can write - it proves the documented algorithm matches the ECU silicon, not just an internet copy of someone else's port.

3.2 - drive the full exchange. The `NodeFactory` and `TestFrame` helpers in `Tests.Unit/TestHelpers/` keep this terse:

```
[Fact]
public void EndToEnd_FixedSeed_CorrectKey_Unlocks()
{
    var algo = new E38Algorithm();
    algo.LoadConfig(JsonSerializer.SerializeToElement(new { fixedSeed = "1234" }));
    var node = NodeFactory.CreateNode(
        module: new Gmw3110_2010_Generic(algo, id: "gm-e38-2byte"));
    var ch = NodeFactory.CreateChannel();

    Service27Handler.Handle(node, new byte[] { 0x27, 0x01 }, ch, nowMs: 0);
    Assert.Equal(new byte[] { 0x67, 0x01, 0x12, 0x34 },
                 TestFrame.DequeueSingleFrameUsdt(ch));

    Service27Handler.Handle(node, new byte[] { 0x27, 0x02, 0x96, 0xCE }, ch, nowMs: 0);
    Assert.Equal(new byte[] { 0x67, 0x02 },
                 TestFrame.DequeueSingleFrameUsdt(ch));
    Assert.Equal(1, node.State.SecurityUnlockedLevel);
}
```

Run them all with `dotnet test Tests.Unit/Tests.Unit.csproj`. Note: testhost reliably reports an "aborted" line after a clean teardown - trust `Failed: 0`, not the exit-code header.

Step 4 - Try it from the UI

1. `dotnet run --project GmEcuSimulator/GmEcuSimulator.csproj` (or `Run` from your IDE).
2. Pick an ECU in the left sidebar.
3. Open the **Setup window** (toolbar button above the PID grid, or Ctrl+Shift+P).
4. Choose `gm-e38-2byte` in the security module dropdown.
5. *(Optional)* Add a config row: `fixedSeed` = `1234` → exchange becomes deterministic.
6. `File > Save`. Confirm the saved `.mode.json` (e.g. `ecu_simulator.mode.json`) shows the current `"Version"`, the ECU's `"SecurityModuleId": "gm-e38-2byte"`, and (if set) `"SecurityModuleConfig": { "fixedSeed": "1234" }`.
7. Connect a J2534 host and walk through `27 01 → 27 02 96 CE`.

If you didn't set a fixed seed, the seed bytes change every request. Recompute the key with `E38Algorithm.ComputeKey(seed)` (or any clone of the C algorithm in the docs) to validate from the tester side.

Step 5 - Going deeper

Multi-level algorithms

The subfunction byte encodes both the *operation* (odd = requestSeed, even = sendKey) and the *level*: `0x01/0x02` = level 1, `0x03/0x04` = level 2, `0x05/0x06` = level 3, etc. Generic does `level = (sub + 1) / 2` automatically. To support more than one level:

```
public IEnumerable<byte> SupportedLevels { get; } = new byte[] { 1, 2 };

public bool ComputeExpectedKey(byte level, ReadOnlySpan<byte> seed, Span<byte> keyBuffer,
out int keyLength)
{
    if (level == 1) return ComputeLevel1Key(seed, keyBuffer, out keyLength);
    if (level == 2) return ComputeLevel2Key(seed, keyBuffer, out keyLength);
    keyLength = 0; return false;
}
```

`SecurityUnlockedLevel` tracks the *highest* level unlocked. The `IsUnlocked(level)` helper a future service can call returns true when `SecurityUnlockedLevel >= level`, so a service requiring level 1 is honoured after a level-2 unlock too.

Different seed / key byte counts

GMW3110-2010 examples are usually 2-byte symmetric, but newer schemes use 3, 4, or 8 bytes; nothing requires `SeedLength == KeyLength`. Set the properties accordingly. The buffers passed in by the generic module are sized off these properties.

Stateful or derived-key algorithms

If your algorithm needs to remember something between requests (e.g. a session salt, a derived intermediate key), use `NodeState.SecurityModuleState` - an opaque `object?` slot the generic module never touches. Cast it back to your own type on each call:

```
public bool ComputeExpectedKey(byte level, ReadOnlySpan<byte> seed, ...)
{
    var mine = ctx.State.SecurityModuleState as MyState ?? new MyState();
    // ... use mine ...
    ctx.State.SecurityModuleState = mine;
}
```

(For the strategy-only path that's harder, since `ISeedKeyAlgorithm` doesn't receive the context. If you need this, drop down to the full `ISecurityAccessModule` - see §7.)

Real-world test vectors

If you log a J2534 tester unlocking a real ECU, you have ground truth. Add the captured pair to your test theory:

```
[InlineData((ushort)0xAABB, (ushort)0x????)]  // captured 2026-XX-XX from VIN ...
```

One captured pair is worth a hundred algorithmic guesses.

What `Gmw3110_2010_Generic` does for you

So you don't reimplement these:

Concern	Behaviour
Malformed payload (wrong length, bad subfunction, `$00` / `$7F` reserved)	NRC `$12` SubFunctionNotSupportedInvalidFormat
Subfunction parity	Odd → requestSeed, even → sendKey
Level derivation	`level = (sub + 1) / 2`, filtered by `algorithm.SupportedLevels`
Already unlocked at this level	Returns seed-all-zero per spec
sendKey without prior requestSeed for the same level	NRC `$22` ConditionsNotCorrectOrSequenceError
Correct key	Sets `SecurityUnlockedLevel`, clears pending state and attempt counter
Wrong key	Increments `SecurityFailedAttempts`, NRC `$35` InvalidKey
Three wrong keys in a row	NRC `$36` ExceededNumberOfAttempts, arms 10s lockout, invalidates pending seed
Request during lockout	NRC `$37` RequiredTimeDelayNotExpired
Lockout deadline elapsed	Counters reset on next request; deadline is a timestamp comparison, no scheduled timer
Thread safety	Per-call lock on `NodeState.Sync` so concurrent J2534 channels don't interleave
P3C activation	`Service27Handler` activates the keepalive window even on NRC responses - failed `$27` is still enhanced traffic

Escape hatch - implementing `ISecurityAccessModule` directly

Skip the `Gmw3110_2010_Generic` envelope entirely when:

- Your algorithm needs more than one round-trip (e.g. challenge → counter-challenge → key)
- Subfunction encoding isn't the standard odd/even pairing
- You want different NRC mappings (e.g. `$33` SecurityAccessDenied in some state)
- You need direct access to the bus to send unsolicited frames during the exchange

In that case, implement `ISecurityAccessModule` yourself:

```
public sealed class MyExoticModule : ISecurityAccessModule
{
    public string Id => "my-exotic";

    public void Handle(SecurityAccessContext ctx)
    {
        // ctx.Node, ctx.Channel, ctx.UsdtPayload, ctx.State, ctx.NowMs
        // ctx.Egress.SendPositiveResponse(subfunction, data);
        // ctx.Egress.SendNegativeResponse(nrc);
        // ctx.Egress.SendRaw(usdtPayload);    // arbitrary bytes
    }

    public void LoadConfig(JsonElement? config) { /* ... */ }
}
```

You're responsible for everything `Gmw3110_2010_Generic` did for free - but you have the whole exchange.

Reference

NRC quick table

Code	Constant	When
`$11`	`Nrc.ServiceNotSupported`	No module configured on the ECU
`$12`	`Nrc.SubFunctionNotSupportedInvalidFormat`	Malformed payload, reserved subfunction, unsupported level
`$22`	`Nrc.ConditionsNotCorrectOrSequenceError`	sendKey without prior matching requestSeed
`$33`	`Nrc.SecurityAccessDenied`	(Module's discretion - generic doesn't emit this; available for custom modules)
`$35`	`Nrc.InvalidKey`	Key didn't match `ComputeExpectedKey`
`$36`	`Nrc.ExceededNumberOfAttempts`	Third wrong key in a row; lockout armed
`$37`	`Nrc.RequiredTimeDelayNotExpired`	Request inside the 10s lockout window

`NodeState` security fields

Field	Reset by power-on	Reset by `$20` ReturnToNormalMode	Reset by successful unlock
`SecurityUnlockedLevel`	✓	✗ (retained per spec)	sets to new level
`SecurityPendingSeedLevel`	✓	✗	✓
`SecurityLastIssuedSeed`	✓	✗	✓
`SecurityFailedAttempts`	✓	✗	✓
`SecurityLockoutUntilMs`	✓	✗	✓
`SecurityModuleState`	✓	✗	not touched

File layout

```
Core/Security/
├── ISecurityAccessModule.cs       // module interface (full-protocol)
├── ISeedKeyAlgorithm.cs           // strategy interface (algorithm only)
├── ISecurityEgress.cs             // egress helpers handed to modules
├── SecurityAccessContext.cs       // ref struct passed to Handle()
├── SecurityModuleBehaviour.cs     // Strict vs BypassAll envelope behaviour
├── SecurityModuleRegistry.cs      // string-id → factory (+ legacy-id remap)
├── Modules/
│   └── Gmw3110_2010_Generic.cs   // bundled envelope module
└── Algorithms/
    ├── E38Algorithm.cs            // worked example (this doc): gm-e38-2byte
    ├── E67Algorithm.cs            // gm-e67-2byte
    ├── T43Algorithm.cs            // gm-t43-2byte
    ├── Gm5ByteAlgorithm.cs        // gm-e92-5byte (DPS 5-byte cipher)
    ├── Gm5BytePasswords.cs        // 256-entry password table for the 5-byte cipher
    └── RandomSeedCipher.cs        // backs the gm-bypass-* permissive modules

Core/Services/
└── Service27Handler.cs            // dispatcher + ChannelEgress wrapper

Tests.Unit/
├── Security/
│   ├── Service27DispatchTests.cs
│   ├── Gmw3110_2010_GenericTests.cs
│   ├── E38AlgorithmTests.cs       // worked example tests
│   ├── E67AlgorithmTests.cs
│   ├── Gm5ByteAlgorithmTests.cs
│   ├── T43AlgorithmTests.cs
│   └── RegistryTests.cs
└── TestHelpers/
    ├── NodeFactory.cs
    ├── FakeSeedKeyAlgorithm.cs
    ├── ThrowingSeedKeyAlgorithm.cs
    └── TestFrame.cs               // PassThruMsg → USDT bytes
```

A note on responsible use

The E38 algorithm has been openly documented in the GM tuning community for over a decade and is included here for **simulator development** - testing J2534 hosts, validating diagnostic flows, exercising the protocol envelope. The simulator is not, and shouldn't be used as, a tool for circumventing security on a vehicle you don't own. Use it on hardware you have rights to.